Theory of Computation-Based Secure API System

Leveraging DFA for Endpoint Security and Predictive Validation

VEER SURENDRA SAI UNIVERSITY OF TECHNOLOGY

Samuel Priyatam Dash
Computer Science & Engineering
5th Semester, 2302040025

Introduction & Objective

The API Security Challenge

Modern applications rely on APIs to exchange sensitive data, making them prime targets for attacks. Traditional security approaches often react to threats rather than prevent them systematically.

A Formal Methods Approach

By modeling API request flows as **Deterministic Finite Automata (DFA)**, we can mathematically guarantee secure state transitions and automatically reject invalid request sequences before they reach application logic.

Project Goal

Automatically validate and secure API endpoints through DFA-driven state management and formal verification

System Design & Workflow

Core Components

Endpoint Definition

Define endpoints with query parameters, expected inputs, and response structures

DFA Validation

Process requests through state machine, validating each transition against security rules

Secure Generation

Generate encrypted responses only after successful state validation

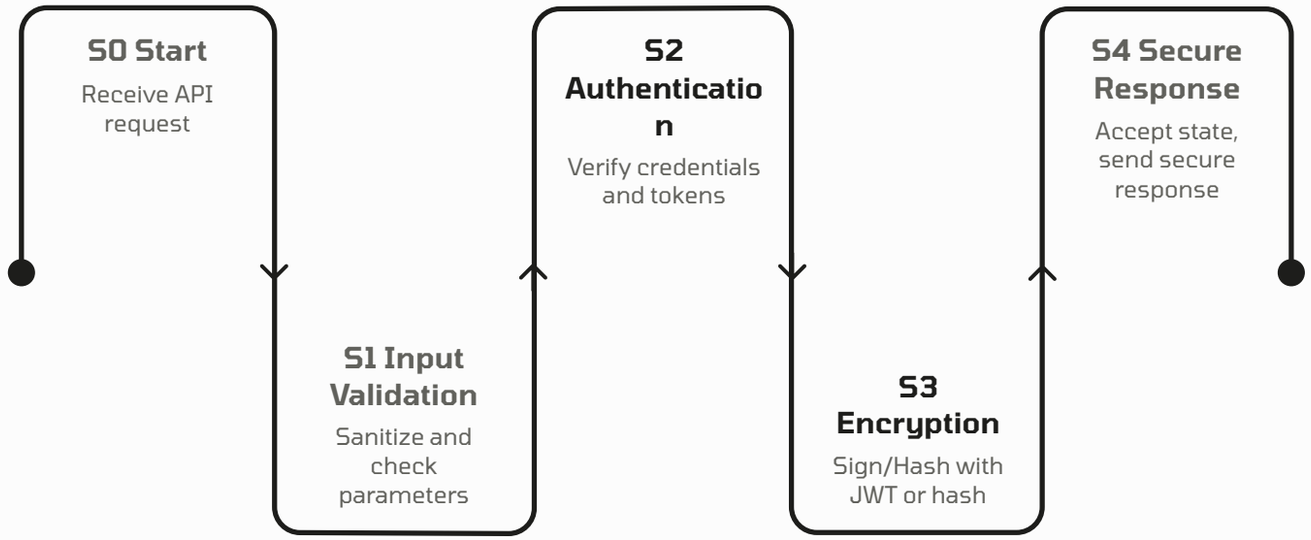
Example Endpoints

- `/api/login` – Authentication entry point
- `/api/dashboard` – Protected user data
- `/api/admin` – Privileged operations

Request Processing

Each request undergoes formal validation: credentials → authentication → encryption → response. Invalid transitions automatically reject the request.

DFA Model & Implementation



State Transition Logic

01

S0: Start State

Initial state receiving API request with credentials and parameters

02

S1: Input Validation

Sanitize inputs, check parameter types, reject malformed requests

03

S2: Authentication

Verify credentials against secure database, enforce access control

04

S3: Encryption

Apply JWT signing or cryptographic hashing to session data

05

S4: Accept State

Generate secure response with encrypted payload and success code

Key Insight: Any invalid input or failed validation immediately transitions to a rejection state, preventing the request from progressing further. This ensures deterministic security enforcement.

Results & Security Analysis

Vulnerability Prevention Through Formal Validation

Insecure API Flow

```
POST /login
{
  "user": "admin' OR '1'='1",
  "pass": "password123"
}
→ Direct DB query
→ SQL injection succeeds
```

Spec snapshot:

```
{
  "body_schema": {
    "password": {
      "type": "string"
    },
    "username": {
      "type": "string"
    }
  },
  "method": "POST",
  "name": "login",
  "path": "/v1/login",
  "query_schema": {
    "id": "sam",
    "password": "sam"
  },
  "response_schema": {
    "status": 202
  }
}
```

DFA-Protected Flow

```
POST /login
{
  "user": "admin' OR '1'='1",
  "pass": "password123"
}
→ S1: Input sanitization
→ Reject: Invalid characters
→ No DB access granted
```

Final secured spec

```
{
  "body_schema": {
    "auth_token": {
      "description": "JWT containing credentials/proof",
      "format": "jwt",
      "type": "string"
    }
  },
  "method": "POST",
  "name": "login",
  "path": "/v1/login",
  "query_schema": {
    "id": "sam",
    "password": "sam"
  },
  "response_schema": {
    "status": 202
  },
  "security_meta": {
    "login_mode": "jwt",
    "rate_limit": {
      "requests_per_min": 30
    },
    "require_https": true
  }
}
```

Security Guarantees

SQL Injection Prevention

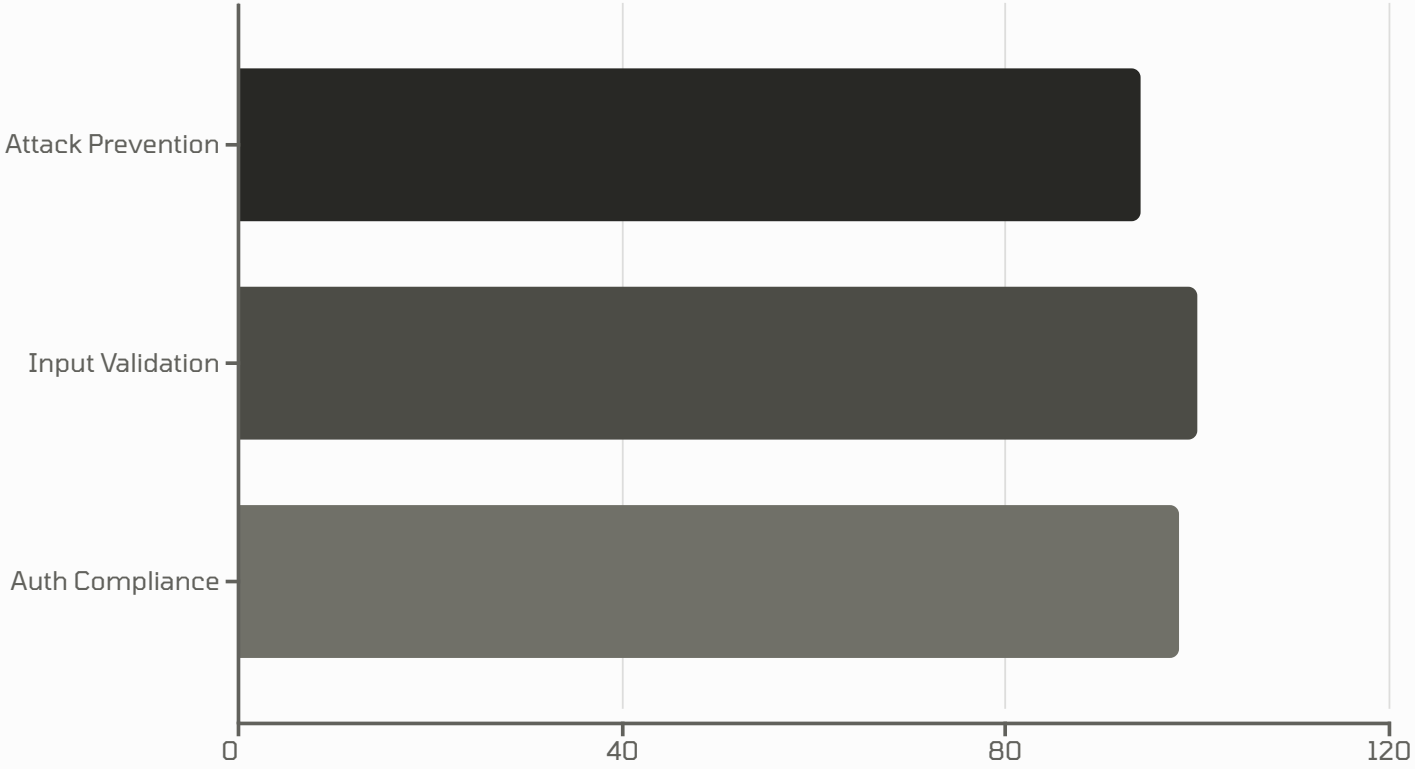
State S1 validates and sanitizes all inputs before any database interaction

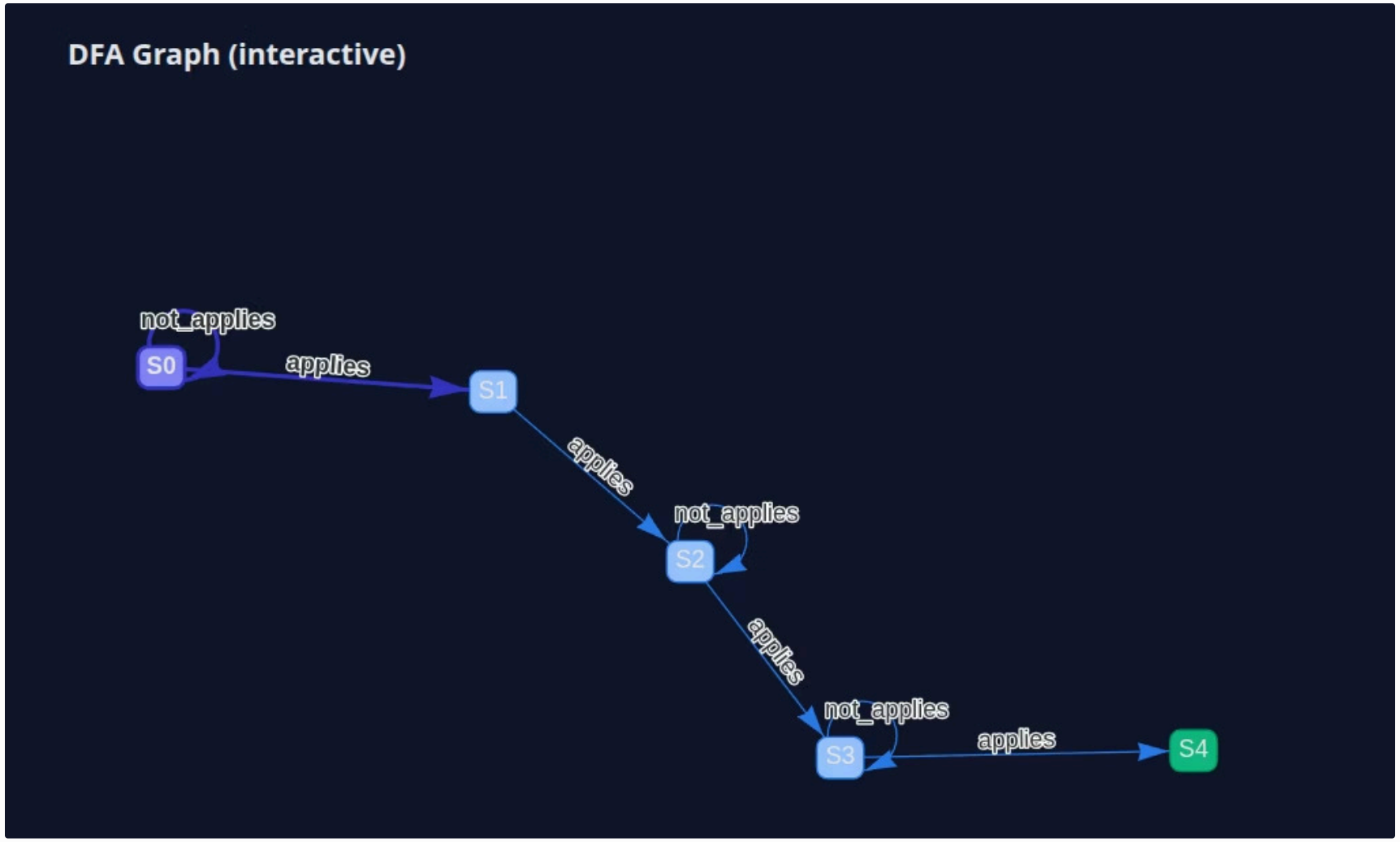
Credential Protection

Passwords never stored or transmitted in plaintext; encryption enforced in S3

Broken Authentication Mitigation

State S2 enforces strict authentication logic before granting access





Sample DFA implementation for login api end point

Conclusion & Future Scope

Key Takeaways

Formal methods provide provable security

DFA-based validation offers mathematical guarantees that traditional security approaches cannot match

Automation reduces human error

State machines systematically enforce security policies without relying on developer vigilance

Theory of computation has practical applications

Abstract automata theory directly translates to real-world API security infrastructure

Future Extensions

Concurrent Session Handling

Implement **Nondeterministic Finite Automata (NFA)** to manage multiple simultaneous API sessions with shared state validation

Adaptive Security States

Integrate **machine learning** to dynamically adjust DFA transitions based on emerging threat patterns and behavioral analysis

> "By grounding API security in formal language theory, we transform reactive defense into proactive, mathematically verifiable protection."